Sound Texture Synthesis Using an Overlap–Add/Granular Synthesis Approach*

MARTIN FRÖJD,** *AES Member*
(froejd@gmail.com)

Chalmers University of Technology, 41296 Göteborg, Sweden

AND

ANDREW HORNER
(horner@cse.ust.hk)

Hong Kong University of Science and Technology, Kowloon, Hong Kong

Texture synthesis, commonly used for graphics, can be applied to sound textures. A real-time capable overlap–add method for sound texture synthesis is described. The method performs significantly better than previous, more complex methods for many types of sounds. It is especially well suited to sounds with a random character. The results suggest that better textures can be achieved using longer scales of granularity than used previously.

0 INTRODUCTION

In computer graphics a texture is a pattern or image that can be repeated (or stretched) to cover a larger area. However, humans can often distinguish repeating patterns easily. To overcome this problem, graphic texture synthesis aims at creating large images that do not seem repetitive—in other words, creating something that seems natural to the human eye.

Sound textures share many of these properties, but they also have some important differences, in particular that a sound texture is repeated in only one dimension whereas graphic textures would be repeated in two dimensions. The basic idea of sound texture synthesis is to be able to produce sounds of any given length that exhibit the same qualities as the source sound. For example, given a 10-second recording of the wind blowing, sound texture synthesis should be able to produce several minutes of sound with the same natural windlike qualities. It should do this without changing the character of the sound, unlike other techniques such as time stretching and rescaling.

Sound textures have a number of useful applications, including sound editing, interactive environments, computer games, and compression. Various applications such as sound tracks for films and advertisements need background environmental sounds of a specified length. If the source sound is too long it can easily be truncated or faded out, but it is not as easily extended. Sound texture synthesis offers a sound editing tool for solving such problems, but the quality of the resulting sound must be very good and without any distracting artifacts.

Interactive environments, such as computer games, are another ideal application for sound texture synthesis. In interactive environments it is not possible to know the length needed for a sound. For example, in a computer game the user might stay in the same environment for several minutes. The standard solution is to loop the sound, and computer game sound designers typically prepare sounds for seamless looping. However, sound textures offer a potentially more dynamic solution, avoiding simple repetition, which can become boring and even annoying. For these real-time applications the sound texture quality has to be good because it is not possible to hide artifacts through postprocessing.

Sound textures also function as a form of compression since instead of a long source sound, only a shorter extendable sample is needed. Many computer games contain several hours of sound, which require considerable data storage [1]. Though sound texture synthesis may produce very natural sounding textures, it is by nature a lossy compression method [2].

*Manuscript received 2008 March 31; revised 2008 December 6.

**Now with Ericsson Ltd., Coventry CV3 IJG, UK.

More controversially, Lu et al. also suggested that sound textures could be used for extending music indefinitely [3]. This would be an especially useful application for disc jockeys, as seen in Jehan's "Automatic DJ" [4].

0.1 Previous Work

There is no established formal definition of a sound texture. Previous work has used different and sometimes conflicting definitions [5]. One of the first attempts at defining sound textures was made by Saint-Arnaud and Popat in 1995, and again in 1998. By their definition, sound textures "exhibit similar characteristics over time, that is, a two-second snippet of a texture should not differ significantly from another two-second snippet." They also note that what differentiates a sound texture from other types of sounds is that the first few seconds of a sound texture contain the essence of the sound, while speech and music present new, unique information continuously over time. As a result a sound texture always contains repetition, sometimes perceivable and sometimes not [6].

Dubnov et al., inspired by two-dimensional texture synthesis, defined a sound texture as being a set of repeating structural elements called sound grains, which locally appear random but globally are similar. They used a statistical approach, modeling sound textures as samplings of stochastic sources. Thus any finite sampling of the source will be similar, but not identical to another sampling from the same source. The samplings share certain properties, and by finding these properties (that is, their mutual source), sound textures can be synthesized with any length. In implementing their definition, they used wavelet tree learning for recombining very short grains. Their method had difficulty with longer sound events and rhythmic sounds, which resulted in artifacts and "stutter" due to the short grains or possibly the lack of enough interpolation [7].

Inspired by Dubnov, Hoskinson and Pai explored a similar granular synthesis method. They were however not content with Dubnov's tree approach and instead used a wavelet-based analysis to find good segmentation points, and then synthesized textures using Markov chain–based recombination. In some cases they used a 5-ms crossfade between their 40-ms (or longer) frames. They reported impressive results as well as real-time performance [8].

Lu et al. used the alternative term "audio texture" and included simple music in its definition. They noted that the most important property of sound textures is their self-similarity. In implementing their theory, they chose a method similar to that of Hoskinson and Pai, but based on 32-ms grains and using a mel-frequency cepstrum coefficient (MFCC) approach borrowed from speech recognition. Their analysis split the sound into subclips, which could then be rearranged to change and extend the sound, producing a sound texture. They achieved some good results, especially when restoring recordings with dropouts, in a way reminiscent of Niedzwiecki and Cisowski's work [3],[9].

Zhu and Wyse departed from the statistical model of Dubnov and instead simplified the original definition by claiming that all the information needed to produce a sound texture can be found within a limited window, assuming the optimal window length is known. Wyse essentially treated sound textures as noisy backgrounds plus Poisson-distributed random events, not unlike the approach of Peltola et al. for synthesizing audiences clapping hands [10]. Zhu and Wyse performed their synthesis based on time- and frequency-domain linear predictive coding—a technique borrowed from the field of speech synthesis and compression—but achieved mixed results [2].

Di Scipio proposed a method for synthesizing environmental sounds without any source, and produced interesting though not completely natural sounding textures [11]. Recht and Whitman focused on synthesizing musical textures using a more complicated statistical approach [12]. Finally Bascou and Pottier used a granularity-based framework for analyzing sound textures [13]. More details of previous sound texture work are summarized in surveys of Strobl et al. [5] and Schwarz [14].

To finalize, the work most closely related to the current study is that of Parker and Behm. They called their method "tiling and stitching," which was inspired by the work of Efros and Freeman on quilted two-dimensional textures [1],[15]. They tiled source blocks together to form textures, using 15% overlaps for block transitions. They also included some interesting work called "chaos mosaic" to iteratively randomize initially ordered sequences of blocks. However, their sound examples exhibited clearly noticeable seams between the blocks.

0.2 Definitions Used

In this study we do not try to define sound textures strictly, but instead propose an approach that is general enough to work for many types of environmental, mechanical, and animal sounds. More controversial examples of sound textures include the cry of an individual baby, the neighing of a horse, and simple music. Our approach requires sound textures to exhibit some self-similarity. It also assumes that for sounds generated by people, animals, and machinery there are many sources rather than just one or two sources (such as a whole nursery of crying babies, rather than just a single crying baby). Overall, this corresponds to most researchers' definitions [5],[16].

Also, when discussing the qualities of sound textures, we define the following types of common artifacts that can appear in sound texture synthesis:

Echo

When two identical sequences of samples are nearly synchronized, the second sequence will sound like an echo of the first. This is an undesirable artifact in sound texture synthesis since it creates a spatial effect rather than adding to the texture.

Cutoff

A cutoff occurs when a sound is truncated unnaturally. For example, if a baby starts crying, a cutoff results when the sound is stopped before the utterance completes.

Sometimes fade-outs can hide cutoffs, though the problem is more apparent with shorter sounds. When there are many successive cutoffs, the result can sound like the source was "put through a blender."

Repetition

When a sound sequence is repeated more or less identically throughout a texture, it will obtrusively stand out. For example, if a certain bird cry is heard many times throughout a texture, it might be perceived as repetitious. Echo can be considered a special case of repetition. Though repetition is the basic principle for extending any texture, it becomes a problem when it is too obvious.

0.3 Analysis of Previous Work

The key factor when creating a sound texture from a source sound is the ratio between the source sound length and the synthesized sound length. If the synthesized length is much longer than the source, repetition is likely to become obvious since there are fewer ways to extract the sound from a short source. For example, synthesizing a 60-second texture from a 4-second source will almost certainly result in obvious repetition. With a short source (for example, less than 10 seconds) it is difficult to produce a natural-sounding texture of a minute or more. On the other hand, a longer source can produce textures lasting for several minutes without obvious repetition.

Researchers have used a variety of time scales to isolate sound events in sound texture synthesis. Most previous work has used relatively short time scales [2],[3],[7],[8],[16]. A significant finding in our work is that longer time scales perform better, even on short source sounds. Obviously the length of the source sound limits the length of the time scale, but we are talking about time scales of a second compared to milliseconds in previous work. These very short time scales can cause artifacts because they cannot preserve higher level characteristics of the sound without complex analysis.

Some researchers have also used longer time scales. Parker and Behm used longer blocks in an approach similar to our overlap–add method [1]. The main difference between their approach and ours is that they used relatively short overlaps between the blocks. This can cause artifacts such as clicks and pops, and the sound examples given on their web site show such artifacts in the seams between the blocks. Their method also requires user intervention to hand-tailor the block lengths for each sound. By contrast, our method is more robust and does not require user intervention.

Most previous work has been based on voice models and music synthesis, but it is disputed whether sound textures have the same general characteristics as voice or music [4],[16]. All previous work has also used an analysis stage, in some cases a very complex analysis. Our method requires no analysis stage. Instead, it takes advantage of the inherent source self-similarity, assuming that many combined random samplings will result in a sound similar to the source. This simplicity makes the algorithm very efficient, with the tradeoff of being less predictable or with less adjustable results.

Regarding efficiency and performance, we found that Dubnov et al.'s wavelet tree learning method uses a $O(n)$ transformation to and from the wavelet domain, whereas the synthesis stage of recombining the grains is, in a naïve implementation, $O(2^n)$. They recommend an optimized implementation, but even this is much slower than overlap–add [7],[8]. Moreover their synthesis stage uses a large amount of memory [17].

Lu et al.'s approach is faster than Dubnov's but still includes a complex analysis stage for matrix construction and MFCC calculation. The synthesis stage is, however, more or less straightforward, depending on the amount of postprocessing required [3].

In other work, Saint-Arnaud and Popat reported that they needed one hour of computation for each second of resynthesized sound [6], but that of course depends on the available computing power. In general, previous work has mostly ignored the issue of performance, probably because sound texture synthesis is still at a relatively formative stage where results are more important than performance.

1 OVERLAP–ADD METHOD

The method of lengthening a sound by manually overlapping segments of the sound is widely used among recording engineers. The challenge is to obscure the seam between two segments, something that would be done in a trial-and-error fashion, using crossfading and different types of filters. As a manual process it is time-consuming and tedious.

This section describes an automated overlap–add method for sound texture synthesis, inspired by this manual technique. The method can be considered a special case of granular synthesis, and also a type of concatenative synthesis [14]. The assumption of this method is that the source clips exhibit self-similarity [3], and by overlap–adding excerpts of the sound clip, we can create a different and longer texture with the characteristics of the original.

The original clip is used as a stochastic source from which blocks are extracted at random locations. The extracted blocks are then overlap–added to the synthesized texture. This process is repeated until the desired length is reached (see Fig. 1).

The overlap–add procedure crossfades pairs of blocks so that as one block fades out the next fades in. When a block completely fades out, another immediately replaces it and begins fading in (see Fig. 2). The blocks are overlapped by 50% of their duration such that two blocks are always crossfading. For the crossfade, simple linear envelopes will work (as shown in Fig. 2), but to achieve a smoother effect, we use a sine envelope to keep the power of the synthesized texture relatively constant. Other envelopes, such as Gaussian or Hann windows, are also possible.

The key parameter for the overlap–add method is the average block size. A short block can only capture short sound events and cuts off longer events, whereas a long

block can capture more complete events but risks increasing the repetitiveness of the synthesized texture. Using empirical studies, we found that most source sounds can use the same average block length and produce very good results, eliminating the need for further analysis. There is also no need for the user to adjust the parameters for different sources, unlike most previous approaches, which have required some degree of user intervention.

The overlap–add process can be thought of as a special case of granular synthesis [18]–[22]. There are two main differences from normal granular synthesis. First, in overlap–add there are never more than two simultaneous grains. Second, the block lengths are much longer—on the order of seconds rather than the 10–100-ms-length grains typically used in granular synthesis. Since granular synthesis usually implies "clouds of grains," meaning many simultaneous grains (perhaps hundreds or thousands of them), the idea of using only two simultaneous grains is not traditional granular synthesis. Likewise, block lengths of 1 second or more push the idea of "grains" to "tree-trunk" proportions, and this is not traditional granular synthesis. However, it is insightful to think of overlap–add as a special case of granular synthesis in a conceptual sense.

The overlap–add method presented here is less derived from granular synthesis and more closely related to the overlap–add component of a phase vocoder, where blocks are crossfaded to resynthesize a music signal. One difference with our overlap–add procedure is that there is no need to deal with phase continuity for two reasons: 1) the blocks are very long, so if artifacts happen, they occur in a low rate, and 2) the slope of the crossfade region is slow enough to avoid clicks or other audible problems.

Other granular synthesis–based texturing methods go to great length to find natural transitions between sounds so that they can be recombined by identifying appropriate sequences of subclips. These methods construct statistical representations of transitions between elements so that new recombinations preserve similar statistical relations to the original. So the question might be asked, how can the new system achieve better results without any analysis? The reason is that the new system preserves the original character of the source better. Traditional granular sound textures usually suffer from the blender effect, when the sound is shredded in uncharacteristic ways. Though some statistical relations are preserved in traditional granular synthesis, other important characteristic relations are lost in the process. As we will show in our results, by taking longer grains, but not too long, these important characteristics seem to be better preserved in a simple but effective way. The relation between preserved characteristics and grain length is potentially complex and should be studied further. Our results, however, give a clear indication what time span is useful when not applying statistical methods.

1.1 Modifications

While the basic overlap–add algorithm performed reasonably well in our preliminary tests, we made some modifications to make the synthesized textures more

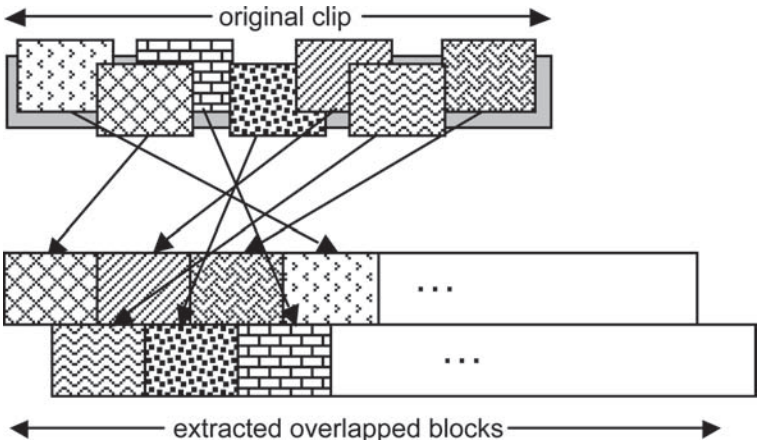

Fig. 1. Extraction of blocks from random locations into overlapped blocks.

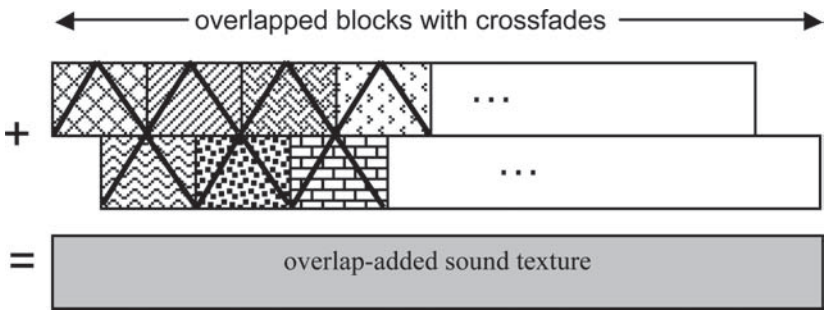

Fig. 2. Crossfading of overlapped blocks to form sound texture.

dynamic. With fixed block lengths, the seams between the blocks always appeared at regular intervals, and periodic artifacts sometimes resulted, which were easy to recognize. To avoid this problem and achieve more dynamic results, we randomized the individual block lengths based on the desired average block length (see Fig. 3). This avoids a regular pattern of seams in the synthesized texture.

However, when creating long synthesized textures from short sources, noticeable repetition can still be a problem, even with randomized block lengths. To minimize this problem, filtering and other effects such as a slight time expansion or compression can be applied to each block to make them seem slightly different. For example, these effects can make a bird call sound like several birds of the same type calling from different locations with slightly different voices [3]. Effects can generally make the sound more dynamic—sounds will sometimes be close by, sometimes far away, sometimes moving, sometimes stationary. In our current implementation the simplest of these effects is included by multiplying each block by a random amplitude scalar, making some blocks more prominent than others. It is a simple idea, but works well.

Moreover, many sounds have particular transients at their beginning and end, such as a characteristic fade-in or fade-out. To capture these characteristics, we simply copy the first and last blocks from the source so that synthesized textures will always begin and end just like their sources. In our current implementation the start and end blocks are set to 1 second each.

As mentioned earlier, when two identical sequences of samples are nearly synchronized, such as when two overlapping blocks take their samples from nearby locations, the second sequence sounds like an echo of the first. This artifact can easily be avoided by simply picking the blocks so as to avoid near synchronization—each block is picked at random and then tested for near synchronization with its predecessor and repicked if needed. However, the impact on the final texture was deemed to be small.

Fig. 4 shows an overview of the complete overlap–add process, including the modifications, and Table 1 summarizes the overlap–add parameters and features used in our investigation.

1.2 Limitations

The overlap–add method assumes that sources are composite rather than simple, meaning that they consist of several smaller sound events. A sound with only few distinguishable sound events—in the simplest case, only one—would not be well synthesized with an overlap–add approach that splits up and overlap–adds the sound with itself. In a sense, overlap–add assumes that sound events are small enough to fit inside (or almost inside) a single block, similar to the approach of Zhu and Wyse [2]. Fortunately simple sounds with few sound events can usually be well synthesized by other means.

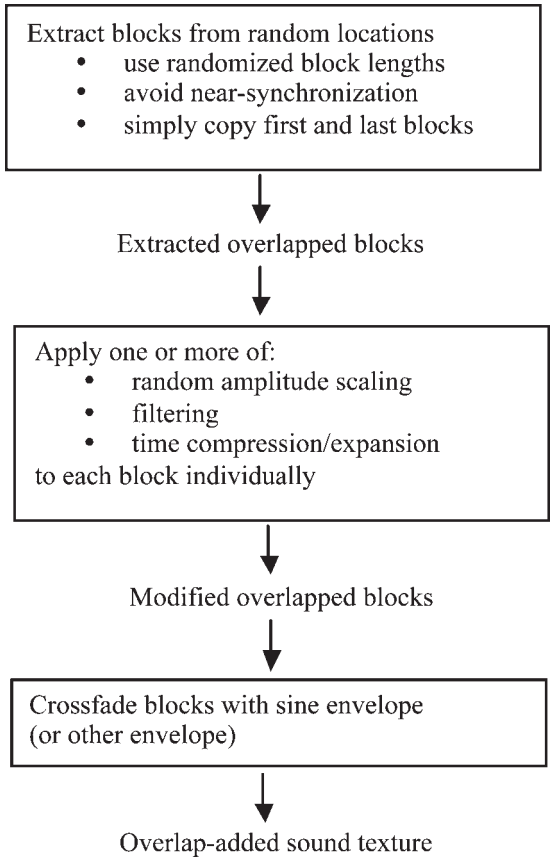

Fig. 4. Overview of overlap–add algorithm.

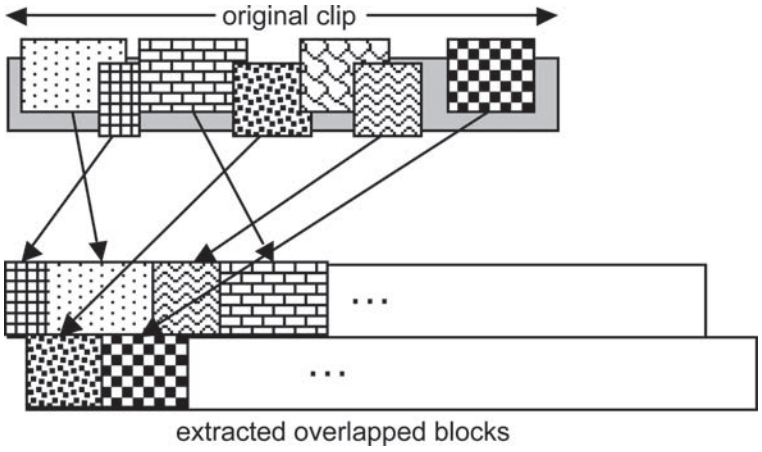

Fig. 3. Extraction of blocks with randomized lengths. Upper row blocks have randomized lengths, while lower row block lengths are set so that there is 50% overlap with upper row blocks.

With overlap–add a sound with a single source will result in a synthesis that sounds like several sources. For example, a recording of a baby crying would result in a texture where more than one baby appears to be crying. This is a result of how the approach works: at any time two separate blocks are overlapped, meaning that one block will be fading in while the other fades out. Thus one baby cry will be emerging while another is dying away. Sometimes the cries might fuse as one; at other times they will sound like two separate cries.

Further this means that overlap–add is not well suited for music or speech. Music usually has a clear underlying rhythm, while speech requires complex casual models of language. Overlap–add cannot easily retain these structures due to the random nature of the algorithm. These types of sounds are better synthesized by other means.

Longer synthesized textures usually require longer source clips to avoid sounding repetitive. We mainly considered source clips ranging from 5 to 15 seconds, and found that source clips lasting 10 seconds or more are needed to synthesize natural sounding textures longer than 1 minute in duration. Judging from the examples provided by previous researchers, most studies have used similar source sound lengths and sound texture lengths. This is probably because most methods would have problems with repetition for a high texture-to-source-length ratio.

We acknowledge that the lack of analysis in our approach can be seen as a limitation. Some of the proposed modifications of the method could also be seen as a form of analysis to overcome other limitations. It is also likely that previous researchers chose to rely on analysis to overcome these limitations. However, as we see in our results, their added analysis has not made much positive difference in terms of the perceived sound texture quality.

2 EVALUATION

To evaluate overlap–add texture synthesis we conducted a listening test. The 22 subjects who participated in the test were students at the Hong Kong University of Science and Technology, ranging in age from 19 to 27 years, and reported no hearing problems. All listeners had at least 5 years of practice on a musical instrument. The subjects were paid for their participation.

The Java command-line program which controlled the experiment was custom written by the authors. Subjects were seated in a "quiet room" with a 40-dB SPL background noise level (mostly due to computers and air conditioning). The stimuli, which were stored on hard disk in 8- and 16-bit integer format, were converted to analog by SoundBlaster X-Fi Xtreme Audio soundcards and then presented through Sony MDR-7506 headphones. The soundcard DAC uses 24 bit with a maximum sampling rate of 96 kHz and a 109-dB signal-to-noise ratio. The sounds were actually played at between 8.0 and 22.05 kHz, depending on the source sound file. The test included eight source clips that were used for synthesis and comparison. Six of the clips were used by earlier studies [3],[7], allowing us to compare methods. See Table 2 for a summary of the source clips and their properties.

At the beginning of the experiment each subject read a set of instructions and was given the opportunity to ask any necessary questions. The order of presentation of the textures was randomized for each subject. Over a period of an hour, each subject was asked to listen to the original and synthesized sound textures.

Test subjects first heard all the original source clips. Then 30-second synthesized textures were presented in a random order. There were four versions of each overlap–add texture with block lengths of 0.25, 0.5, 1, and 2 seconds. Subjects rated each texture for its naturalness (see Table 3 for the rating scheme). A high score of 7 meant that the texture sounded as natural as the original clip.

There are two main reasons why a sound texture may sound unnatural: 1) the texture might sound unnaturally "chopped up" as if it were "put through a blender," and 2) the texture might sound unnatural due to too much repetition. Short blocks are more likely to cause the first problem, whereas long blocks are more likely to cause the second. The goal is to find the stable middle ground where both these artifacts are avoided. In any case the important point is to use rather long textures in the evaluation so that these problematic artifacts can clearly show up since both these problems become more apparent when the texture is longer. This is why we chose to test relatively long textures of 30 seconds, which is about 2–3 times longer than the textures tested by Dubnov and Lu.

In our experiments users were asked to rate four trials for each texture, where each trial was set using a different

Table 1. Parameters/features used for overlap–add method in investigation.

Parameter/Feature	Description
Average block length	Lengths of 0.25, 0.5, 1.0, and 2.0 s were tested
Block = length deviations	Individual block lengths were randomized within ±50% of average block length
Block overlaps	50% of block length for even-numbered blocks (Fig. 3)
Crossfade	Sine envelope (i.e., first 180° of sine period)
Effects	Random amplitude scaling
Transient handling	First and last blocks were retained from source sound
Near synchronization	Avoided by repicking blocks where offset between overlapped blocks was less than *BLOCK_LENGTH/3.6*

random seed. Users also rated textures produced by Dubnov and Lu (four textures produced by Dubnov and two by Lu, as shown in Table 2). In total each subject listened to eight source clips, six textures produced by Dubnov/Lu, and 128 textures produced by our overlap–add method. The original and synthesized textures can be found in [23].

3 RESULTS

The data produced by the listening test allowed us to determine both the optimal average block length for overlap–add and the overall performance of the methods. The listening test data were analyzed to see whether there were any extreme outliers, but none were found. The average rating was calculated for each source sound and block length, along with the average ratings of Dubnov/Lu's textures where applicable (see Table 4).

Table 2. Details of source clips and synthesized textures used in listening test.

Sound Clip Name	Sample Rate (Hz)	Bits per Sample	Length (seconds)
Source sounds			
aviary.wav	22 000	8	9.00
baby.wav	11 025	16	13.67
racing.wav	11 025	8	16.62
rain.wav	22 050	16	2.72
seagulls.wav	22 000	8	14.63
shore.wav	11 025	8	18.13
stream.wav	8000	16	4.17
trafficjam.wav	11 025	8	22.28
Dubnov/Lu synthesized textures			
baby.wav (Dubnov)	11 025	16	11.89
racing.wav (Dubnov)	11 025	16	23.78
rain.wav (Lu)	22 050	16	13.12
shore.wav (Dubnov)	11 025	16	11.89
stream.wav (Lu)	8000	16	12.27
trafficjam.wav (Dubnov)	11 025	16	35.67
Overlap–add synthesized textures			
aviary.wav	22 000	8	30.0
baby.wav	11 025	16	30.0
racing.wav	11 025	8	30.0
rain.wav	22 050	16	30.0
seagulls.wav	22 000	8	30.0
shore.wav	11 025	8	30.0
stream.wav	8000	16	30.0
trafficjam.wav	11 025	8	30.0

Table 4 shows that longer block lengths generally gave better results. In our own informal tests even longer block lengths of 4 seconds also gave acceptable results, though there was more repetition. There are obvious problems when the block length approaches the length of the source sound, since there are fewer ways to segment the source.

Blocks of 2 seconds seem the best balance of these factors, producing an average score of 4.75, corresponding to a rating of "mostly natural." By comparison, the average score for Dubnov/Lu taken together was 3.16, corresponding to a rating of "somewhat unnatural." (The two textures of Lu received an average score of 3.79, compared to 2.85 for the four textures of Dubnov.) We assume that the sound textures provided by Dubnov and Lu were those they judged to be best for various parameter settings.

The most impressive improvements were for the shore and racing car examples. Dubnov's synthesized textures of these examples scored the lowest of the six examples, while for overlap–add they scored the highest. It makes sense that overlap–add would do well on sounds that have a quasioscillatory character, such as waves washing on the shore and racing cars periodically screaming by.

Overlap–add using 2-second block lengths produced better textures than the methods of Dubnov and Lu for almost all source clips (only Lu's rain texture was better, and only by a small margin). Interestingly, on the shortest source sounds (rain and stream) overlap–add produced sound textures roughly equal to those of Lu, while using considerably longer time scales. This indicates that shorter time scales do not necessarily perform better on short source sounds.

Paired sample t-tests [24] revealed that the average score for overlap–add was significantly higher than that for the six synthesized textures of Dubnov and Lu taken together. We compared against both source clips ($t = 2.46$, $p < 0.001$) and listeners ($t = 8.53$, $p < 0.001$).

4 CONCLUSIONS

There are many different kinds of sound textures, and no single algorithm will be best on all of them. We have investigated an overlap–add method for sound texture

Table 3. Rating scheme for listening test.

Score	Meaning
1	Very unnatural
2	Mostly unnatural
3	Somewhat unnatural
4	Somewhat natural
5	Mostly natural
6	Very natural
7	As natural as original

synthesis, which can be considered a special case of granular synthesis with the nontraditional attributes of long grains and no more than two simultaneous blocks. The method produces consistently natural sounding textures for environmental, mechanical, and animal sounds. It is especially well suited to those sounds with a quasioscillatory or random character such as waves splashing on the shore and traffic noise. It also works well for animal and mechanical sounds that have several sources such as a group of chattering birds. These are important classes of background sounds for computer games, films, and advertisements.

Our listening test results show that overlap–add texture synthesis produces mostly natural sounding results when block lengths are 1 second or more. They also indicate that overlap–add performs significantly better than the methods of Dubnov and Lu for the test bed of environmental and mechanical sounds. The results indicate that longer blocks perform better than the shorter blocks used by previous researchers while still using similar source sound lengths. To further verify the results, future work should perform an objective comparison of sound textures using, for example, autocorrelation or spectral analysis, but that is outside the time scope of this study.

In addition overlap–add is very efficient with a time complexity of $O(n)$, since the method does not rely on an analysis step (unlike other methods). Overlap–add is capable of real-time performance in computer games and other interactive environments, which is a useful advantage.

Regarding the parameters of overlap–add, it is already noted that longer blocks create more natural sounding textures. However, this correspondence is not simply linear and should be further studied, both theoretically and empirically. The same applies to the other parameters relevant for overlap–add.

It is also recognized that further processing and analysis could improve the output, such as using filters and effects to help minimize obvious repetition. Low-pass filters, optimized envelopes, and pitch shaping are all potentially useful at the cost of slightly more computation. Also a segmentation technique could be used to optimize the individual block lengths [3], [25], though the expected gain would be relatively small. Finally to make this approach suitable for professional use, stereo channel support in the algorithm could be added.

5 ACKNOWLEDGMENT

The authors wish to thank Jenny Lim and Lee Chung for their invaluable work in the preparation and administration of the listening tests. This work was supported in part by the Hong Kong Research Grant Council's projects HKUST613806 and HKUST613508.

REFERENCES

[1] J. R. Parker and B. Behm, "Creating Audio Textures by Example: Tiling and Stitching," in *Proc. IEEE Int. Conf. on Acoustics, Speech, and Signal Processing, (ICASSP '04)*, vol. 4 (2004 May), pp. 17–21.

[2] X. Zhu and L. Wyse, "Sound Texture Modeling and Time–Frequency LPC," in *Proc. 7th Int. Conf. on Digital Audio Effects (DAFx'04)*, pp. 345–349.

[3] L. Lu, L. Wenyin, and H. J. Zhang, "Audio Textures: Theory and Applications," *IEEE Trans. Speech Audio Process.*, vol. 12, pp. 156–167 (2004 Mar).

[4] T. Jehan, "Creating Music by Listening," Ph.D. dissertation, Massachusetts Institute of Technology, Cambridge, MA (2005), pp. 106–108.

[5] G. Strobl, G. Eckel, and D. Rocchesso, "Sound Texture Modeling: A Survey," in *Proc. Sound and Music Computing Conf. (SMC'06)* (Marseilles, France, 2006).

[6] N. Saint-Arnaud and K. Popat, "Analysis and Synthesis of Sound Textures," in *Computational Auditory Scene Analysis* (Lawrence Erlbaum, Mahwah, NJ, 1998), pp. 293–308.

[7] S. Dubnov, Z. Bar-Joseph, R. El-Yaniv, D. Lischinski, and M. Werman, "Synthesizing Sound Textures through Wavelet Tree Learning," *IEEE Computer Graphics and Appl., Virtual Worlds, Real Sounds*, pp. 38–48 (2002 July/Aug).

[8] R. Hoskinson and D. Pai, "Manipulation and Resynthesis with Natural Grains," in *Proc. Int. Computer Music Conf. (ICMC)*, (Havana, Cuba, 2001), pp. 338–341.

Table 4. Average scores for different methods and block lengths.

		Overlap-add Score with Various Block lengths			
Source Clip	Dubnov/Lu Score	0.25 s	0.5 s	1.0 s	2.0 s
aviary	N/A	3.70	3.96	4.42	4.44
baby	4.09	2.21	3.19	3.96	4.62
racing	1.59	2.12	3.48	4.65	5.36
rain	3.95	3.86	3.85	3.89	3.90
seagulls	N/A	4.06	4.51	4.47	4.73
shore	2.27	4.59	5.47	5.88	5.78
stream	3.63	4.45	4.06	4.05	4.17
trafficjam	3.45	3.17	3.87	4.63	5.00
All	3.16	3.52	4.05	4.50	4.75

[9] M. Niedzwiecki and K. Cisowski, "Smart Copying — A New Approach to Reconstruction of Audio Signals," *IEEE Trans. Signal Process.*, vol. 49, pp. 2272–2282 (2001).

[10] L. Peltola, C. Erkut, P. R. Cook, and V. Valimaki, "Synthesis of Hand Clapping Sounds," *IEEE Trans. Audio, Speech, Language Process.*, vol. 15, pp. 1021–1029 (2007).

[11] A. Di Scipio, "Synthesis of Environmental Sound Textures by Iterated Nonlinear Functions," in *Proc. 2nd COST g-6 Workshop on Digital Audio Effects (DAFX'99)*, (Trondheim, Norway, 1999).

[12] B. Recht and B. Whitman, "Musically Expressive Sound Textures from Generalized Audio," in *Proc 6th Int. Conf. on Digital Audio Effects (DAFx'03)*, (London, UK, 2003).

[13] C. Bascou and L. Pottier, "GMU, A Flexible Granular Synthesis Environment in MAX/MSP," in *Proc. Sound and Music Computing Conf. (SMC'05)*, (Salerno, Italy, 2005).

[14] D. Schwarz, "Concatenative Sound Synthesis: The Early Years," *J. New Music Res.*, vol. 35, pp. 3–22 (2006).

[15] A. A. Efros and W. T. Freeman, "Image Quilting for Texture Synthesis and Transfer," in *Proc. 28th Ann. Conf. on Computer Graphics and Interactive Techniques (SIGGRAPH '01)* (ACM Press, New York, 2001), pp. 341–346.

[16] M. Athineos and D. P. W. Ellis, "Sound Texture Modeling with Linear Prediction Both in Time and Frequency Domains," in *Proc. Conf. on Acoustics, Speech and Signal Processing* (*2003*).

[17] Z. Bar-Joseph, R. El-Yaniv, D. Lischinski, and M. Werman, "Texture Mixing and Texture Movie Synthesis Using Statistical Learning," *IEEE Trans. Visualization Computer Graph.*, vol. 7, pp. 120–135 (2001 Apr.–Jun).

[18] I. Xenakis, *Formalized Music* (Indiana University Press, Bloomington, IN, 1971).

[19] C. Roads, "Automated Granular Synthesis of Sound," *Computer Music J.*, vol. 2, no. 2, pp. 61–62 (1978).

[20] C. Roads, "Granular Synthesis of Sound," in *Foundations of Computer Music*, C. Roads and J. Strawn, Eds. (MIT Press, Cambridge, MA, 1985), pp. 145–159.

[21] C. Roads, "Asynchronous Granular Synthesis," in *Representations of Musical Signals*, G. DePoli, A. Piccialli, and C. Roads, Eds. (MIT Press Cambridge, MA, 1991), pp. 143–186.

[22] C. Roads, *The Computer Music Tutorial* (MIT Press, Cambridge, MA, 1996), pp. 168–184.

[23] http://www.cse.ust.hk/∼horner/soundtextures/.

[24] R. V. Hogg and J. Ledolter, *Applied Statistics for Engineers and Physical Scientists* (Maxwell Macmillan, New York, 1992).

[25] J. Foote and M. Cooper, "Media Segmentation Using Self-Similarity Decomposition," *Proc. SPIE Storage and Retrieval for Multimedia Databases*, vol. 5021, pp. 167–175 (2003).

THE AUTHORS

M. Fröjd

Martin Fröjd received an M.Sc. degree in software engineering from the Chalmers University of Technology, Gothenburg, Sweden. The work described in this paper was part of his Master's thesis, performed during an exchange and internship at the Hong Kong University of Science and Technology. He is now a software engineer with Ericsson Ltd., Coventry, UK.

A. Horner

Andrew Horner received a Ph.D. degree in computer science from the University of Illinois at Urbana-Champaign.

He is a professor in the Department of Computer Science at the Hong Kong University of Science and Technology. His research interests include music synthesis, musical acoustics of Asian instruments, peak factor reduction in musical signals, music in mobile phones, and spectral discrimination.